Acadrix

Student Performance Prediction System

Predict a student's final score early, and turn it into a plan.



Project 1 · AIML Summer Internship 2026 · IIHMF, MNNIT Allahabad

Mohammad Farhan · Tanish Yadav · Garima Jaiswal

AGENDA

A walkthrough of the full machine learning pipeline

01 **Introduction**
Problem statement and objectives

02 **Data & Methodology**
1,015 student records and the six-step pipeline

03 **Preprocessing & EDA**
Cleaning, charts and feature selection

04 **Models & Results**
Three models compared; the best one deployed

05 **Demo & Conclusion**
The live app, conclusions and future scope

PROBLEM STATEMENT

Struggling students are spotted only when results arrive

The warning signs are there all semester: falling attendance, missed assignments, weak internal marks. Colleges record them but rarely act on them in time.

Goal: predict each student's final score (0–100) part-way through the semester, so teachers can step in early and students can change course.

Every prediction maps to a risk level

Low risk
65 and above · on track

Medium risk
50–64.9 · needs support

High risk
below 50 · act now

OBJECTIVES

Six goals, from data to a working app

01

Predict the final score from five academic indicators

02

Follow the full ML lifecycle, from data to deployment

03

Train and compare three models, deploy the best

04

Flag at-risk students for teachers before exams

05

Give every student a personal, actionable plan

06

Deploy a web app that takes inputs and shows predictions

DATASET DESCRIPTION

1,015 student records, deliberately messy

1,015
rows

12
columns

Generated by a seeded script so results are reproducible. Hidden ability and motivation traits drive each student, so features correlate like real academic data.

Built-in mess to clean: about 2% missing values, 15 duplicate rows, 8 impossible values.

Column	Range
Attendance %	0 – 100
Assignment score	0 – 100
Internal assessment average	0 – 100
Previous semester score	0 – 100
Study hours per week	0 – 30
Sleep, commute, gender, extracurricular, internet	candidates
Final score (target)	**0 – 100**

METHODOLOGY

Six steps, from raw data to a live app

STEP 1

Collect

Generate 1,015 seeded student records

STEP 2

Clean

Duplicates, impossible values, missing values, encoding

STEP 3

Explore

Histograms, boxplots, scatter plots, heatmap

STEP 4

Select features

Keep |r| ≥ 0.15; test engineered features

STEP 5

Train and compare

3 models, 80/20 split, 5-fold cross-validation

STEP 6

Deploy

Best model served by Flask, live on Render

DATA PREPROCESSING

Cleaning took 1,015 raw rows down to 992 usable ones

1,015
raw rows

→

−15
duplicate rows

→

−8
impossible values

→

992
clean rows

79 gaps filled

Median for numbers, most common value for categories

3 columns encoded

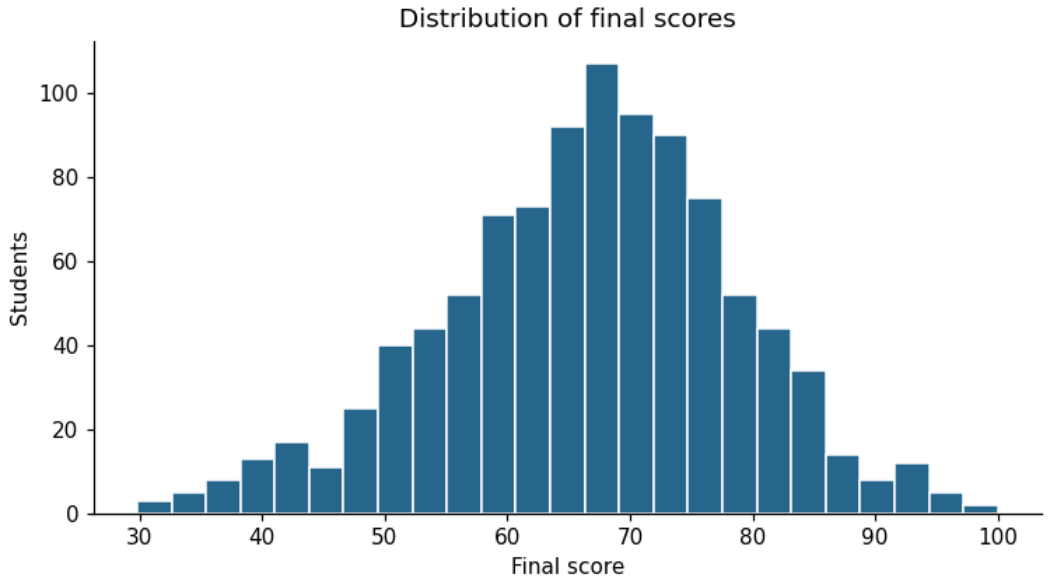
yes/no and gender turned into 0/1

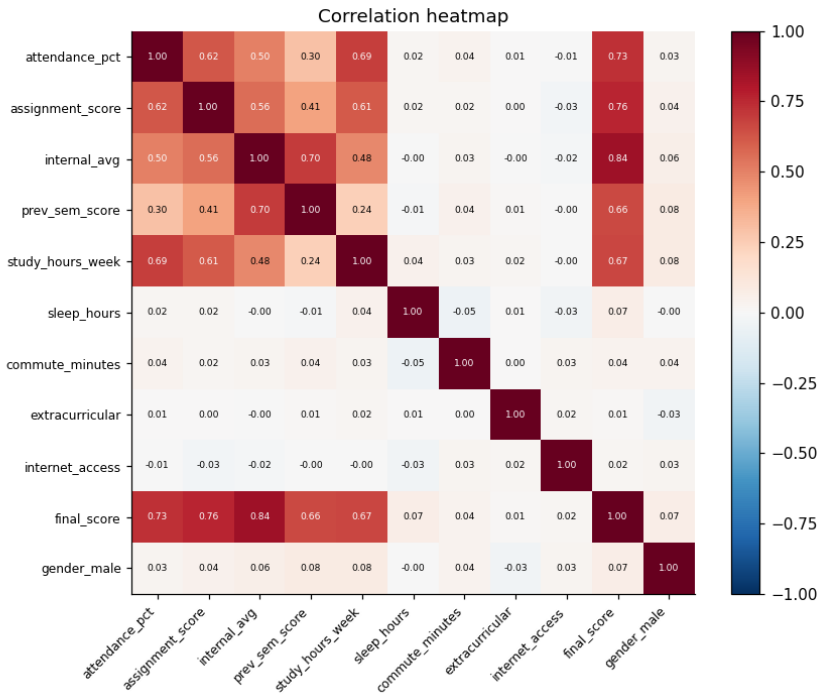
Scaled when needed

StandardScaler for Linear Regression only

EXPLORATORY DATA ANALYSIS

Internal marks track the final score most closely





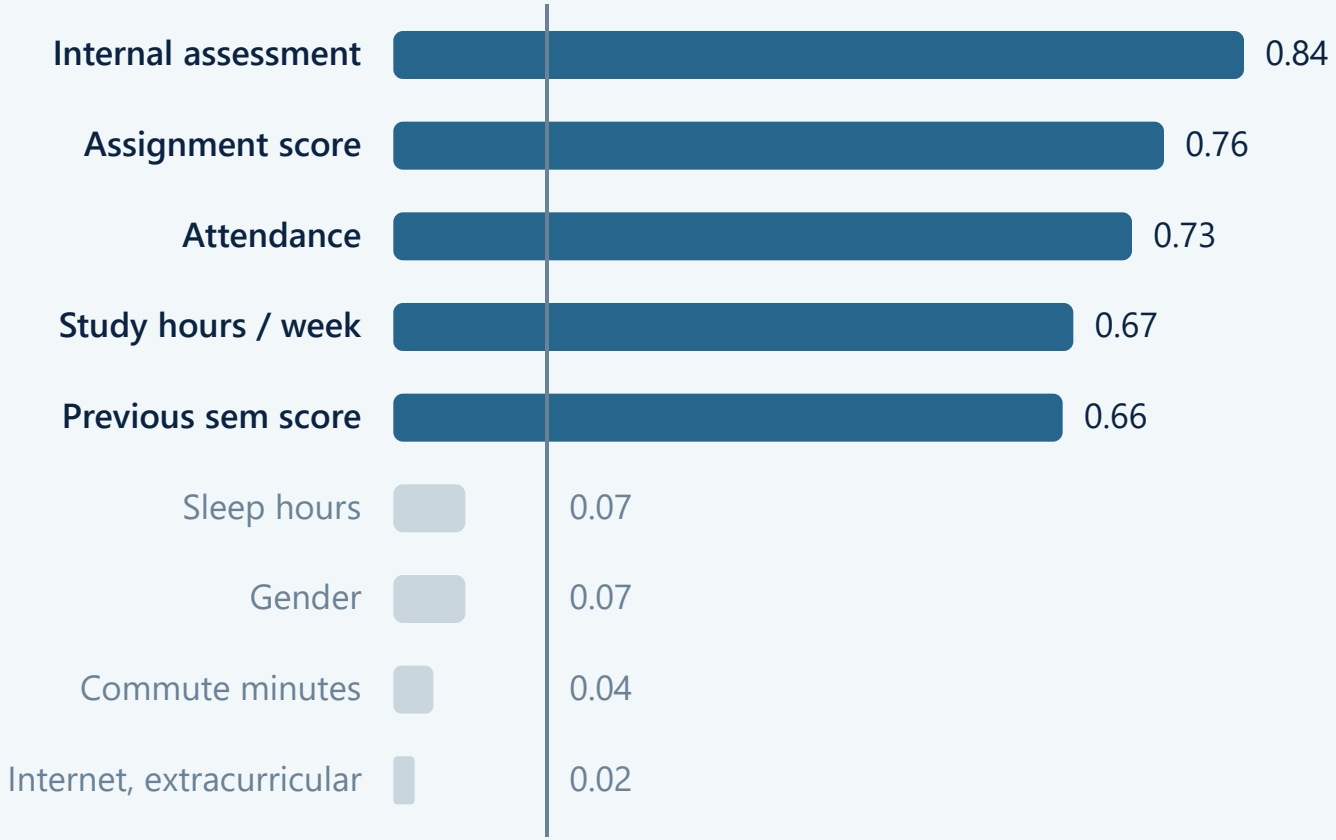
Univariate: scores average 66.6 (range 30–100); most students sit between 55 and 80.

Correlation: internal assessment r = 0.84; sleep, commute and gender all under 0.08.

FEATURE ENGINEERING AND SELECTION

Five academic features kept, five lifestyle columns dropped

Correlation with final score (r) · keep if r ≥ 0.15



Engineered features tested

Engagement index
(attendance + assignment) / 2
CV R^2 change: −0.0065 · dropped

Study efficiency
internal ÷ (study hours + 1)
CV R^2 change: −0.0033 · dropped

Kept only if they raised cross-validated R^2 by more than 0.005.

MODELS USED

One model from each family, trained on the same data

STATISTICAL

Linear Regression

Score = a weighted sum of the five inputs, with weights that minimise squared error.

Settings: StandardScaler + ordinary least squares

BAGGING ENSEMBLE

Random Forest

Averages many decision trees, each trained on a random sample of rows and features.

Settings: 300 trees, min 3 samples per leaf

BOOSTING ENSEMBLE

XGBoost

Builds trees one after another, each fixing the errors the earlier trees still make.

Settings: 400 trees, depth 4, learning rate 0.05, monotone constraints

Evaluated on a held-out 20% test set (199 students) with MAE, MSE, RMSE and R^2, plus 5-fold cross-validation.

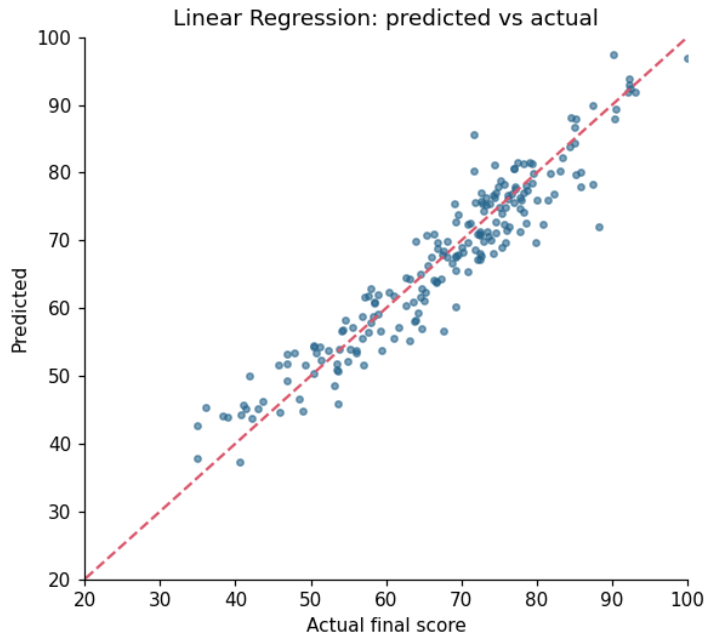
RESULTS

Linear Regression wins narrowly: R^2 0.901, error ±3.3 marks

Model	MAE	RMSE	R^2	CV R^2
Linear Regression	**3.32**	**4.17**	**0.901**	**0.893**
Random Forest	3.24	4.26	0.897	0.879
XGBoost	3.33	4.28	0.896	0.889

Why the simplest model won

The score is close to a weighted sum of the inputs plus random noise, so the tree models find little curve to learn. What error is left is mostly that noise.



Points follow the diagonal; the lowest scorers are predicted a little high.

DEMO SCREENSHOTS

Teachers see who needs help; students get a plan



Teacher dashboard: class statistics and students flagged high risk



Student overview: score, grade, risk, rank



Goal planner: two ways to reach a target

CONCLUSION

Five indicators predict the final score to within ±3.3 marks

0.901

R^2 on 199 unseen students

8 / 8

capstone phases completed, data to deployment

Live

two-portal web app on Render

The lesson: simpler was better here

Random Forest and XGBoost matched Linear Regression but did not beat it. The simplest model is the fastest, the most stable and the easiest to explain to a teacher.

Findings agree with published research: internal assessment and prior achievement are the strongest predictors, and attendance matters.

FUTURE SCOPE

Next: real data and permanent hosting

Real student records

Retrain on institutional data with the same five fields

Better for weak students

Fix the over-prediction for the lowest scorers

Permanent storage

Paid hosting or PostgreSQL so data survives restarts

ERP integration

Import attendance and marks automatically

Smarter Drix

Connect a language model for free-form questions

Explainable AI

Per-student explanations such as SHAP values

THANK YOU

Questions & Discussion

Code: github.com/Garima1223/acadrix-student-performance

Live app: acadrix-da7i.onrender.com

